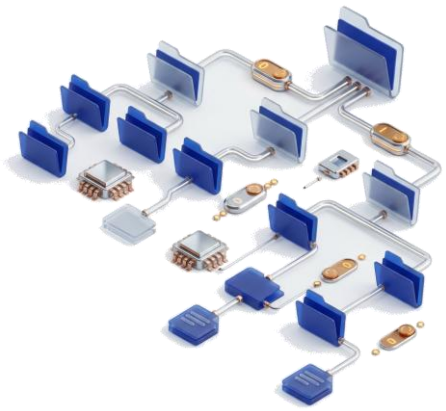
ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
KHOA KHOA HỌC MÁY TÍNH

BÁO CÁO ĐỒ ÁN HỌC PHẦN
LẬP TRÌNH HỆ THỐNG (SYSTEM PROGRAMMING)

ĐỀ TÀI 17:
NGHIÊN CỨU HỆ THỐNG TỆP LINUX VÀ TỰ CÀI ĐẶT LỆNH ls VỚI CÁC TÙY CHỌN -a, -l, -R, -S BẰNG NGÔN NGỮ C (POSIX.1-2008)



Mô hình kết nối giữa Cây thư mục (Dentry) và Siêu dữ liệu Inode trong Linux VFS

Thông tin Đồ án	Chi tiết
Sinh viên thực hiện:	Nguyễn Xuân Lĩnh
Học phần:	Lập trình Hệ thống (System Programming)
Đề tài:	Đề tài 17 – Implement ls with -a, -l, -R, -S
Môi trường thực nghiệm:	Ubuntu 24.04 LTS (WSL2 Kernel 6.18) · GCC 13.3.0 · GNU Make 4.3
Kho mã nguồn GitHub:	https://github.com/Xlinhx/DoAn_LapTrinhHethong

Đà Nẵng – Năm 2026

CẤU TRÚC BÁO CÁO VÀ NGUYÊN TẮC TRÌNH BÀY

Báo cáo này được biên soạn theo phương pháp Chuỗi Nhân - Quả Đánh Số Tuần Tự (1 -> 2 -> 3...). Thay vì chép lại lịch sử hệ điều hành Linux hay lý thuyết chung chung, báo cáo tập trung giải thích trực diện từ bản chất gốc rễ đến từng quyết định kỹ thuật trong mã nguồn: Tại sao cơ chế A của hệ điều hành lại bắt buộc chương trình phải dùng giải pháp B.

Phần	Tên Chương	Nội dung Trọng tâm
Chương 1	Tổng quan Đề tài 17 & Phân tích Yêu cầu Kỹ thuật	Phạm vi 4 cờ -a, -l, -R, -S; tại sao cấm dùng system('ls'); tiêu chí nghiệm thu POSIX.
Chương 2	Cơ sở Lý thuyết Hệ thống Tệp Linux (Từ Gốc đến Ngọn)	Giải thích tuần tự 1 -> 4: Bản chất Dentry vs Inode, Hard link vs Symlink, giải phẫu 16-bit st_mode và bảng ánh xạ POSIX APIs.
Chương 3	Thiết kế Kiến trúc & 8 Quyết định Kỹ thuật Trọng tâm	Cấu trúc dữ liệu động EntryList, luồng chạy 16 bước tuần tự và giải thích chi tiết 'Tại sao A phải B' cho từng module C.
Chương 4	Thực nghiệm trên Ubuntu WSL2, Đối chiếu GNU ls & Đánh giá	Giải thích tại sao kiểm thử trên /tmp thay vì /mnt/p; kết quả 15 test cases tự động; kiểm tra bộ nhớ ASan/LSan; giới hạn và hướng mở rộng.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 17 & PHÂN TÍCH YÊU CẦU KỸ THUẬT

1.1. Đặt vấn đề: Tại sao phải tự cài đặt lệnh ls bằng POSIX C?

1. Sự khác biệt giữa gọi lệnh vỏ (Shell Wrapper) và Lập trình Hệ thống: Lệnh ls (list directory contents) là công cụ nền tảng nhất trên mọi hệ điều hành họ UNIX/Linux. Tuy nhiên, dưới góc độ Lập trình Hệ thống, việc gọi hàm system("ls -alRS") trong C chỉ là tạo một tiến trình con nhờ chương trình có sẵn của hệ điều hành chạy hộ, hoàn toàn không thể hiện sự làm chủ đối với Hệ thống tệp ảo (VFS), Bảng mô tả tệp (File Descriptor Table) hay cấu trúc Inode của nhân Linux.

2. Mục tiêu cốt lõi của Đề tài 17: Do đó, Đề tài 17 đặt ra yêu cầu xây dựng mới hoàn toàn chương trình thực thi ./myls bằng ngôn ngữ C (chuẩn C11 + POSIX.1-2008), giao tiếp trực tiếp với nhân hệ điều hành thông qua các lời gọi hệ thống (System Calls) và thư viện C chuẩn để tái hiện chính xác hành vi của lệnh ls với 4 tùy chọn cốt lõi: -a, -l, -R, và -S.

1.2. Phân tích chi tiết 4 tùy chọn chức năng (-a, -l, -R, -S)

Tùy chọn	Yêu cầu Hiển thị & Hành vi	Ràng buộc Kỹ thuật Bắt buộc

-a (All)	Hiển thị tất cả các mục trong thư mục, bao gồm cả các tệp và thư mục ẩn bắt đầu bằng dấu chấm ('.', '..', '.env', '.gitignore').	Mặc định khi không có -a, chương trình phải lọc bỏ mọi mục có d_name[0] == '.' ngay khi đọc từ readdir().
-l (Long format)	Hiển thị danh sách chi tiết trên từng dòng: Loại tệp + 9 bit quyền + bit đặc biệt (SUID/SGID/Sticky), số hard link, tên Owner, tên Group, kích thước (bytes), dấu thời gian sửa đổi (mtime) và đích trỏ tới của Symlink.	Phải in dòng 'total <1K-blocks>' ở đầu mỗi thư mục và tự động tính độ rộng lớn nhất của từng cột để căn lề thẳng hàng.
-R (Recursive)	Duyệt đệ quy toàn bộ cây thư mục con nhiều tầng theo thứ tự đã sắp xếp.	Phải liệt kê hết toàn bộ mục của thư mục hiện tại trước rồi mới đi xuống thư mục con; bắt buộc chặn '.', '..' và Symlink để không lặp vô hạn.
-S (Sort by Size)	Sắp xếp các mục theo kích thước tệp (st_size) giảm dần (từ lớn nhất đến nhỏ nhất).	Tránh tràn số 64-bit off_t khi so sánh trong qsort(); nếu 2 tệp có cùng kích thước thì phân định thứ tự phụ (tie-break) theo tên từ A đến Z.
Kết hợp tự do	Cho phép người dùng kết hợp các cờ theo bất kỳ thứ tự nào: -al, -lS, -aR, -lR, -alRS hoặc truyền nhiều đường dẫn hỗn hợp.	Dùng getopt() chuẩn POSIX để phân tích cờ; tách riêng nhóm tệp đơn lẻ (in trước) và nhóm thư mục (in sau).

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT HỆ THỐNG TỆP LINUX (GIẢI THÍCH TUẦN TỰ TỪ GỐC ĐẾN NGỌN)

2.1. Chuỗi lý giải nền tảng 1 -> 4: Tại sao đọc thư mục lại cần cả readdir() lẫn lstat()?

Số 1 – Bản chất của Thư mục (Directory) chỉ là bảng ánh xạ Tên -> Số Inode: Hãy hình dung hệ thống tệp Linux giống như một phòng quản lý nhân sự. 'Thư mục' (Directory) không phải là cái hộp chứa toàn bộ hồ sơ của tệp, mà thực chất chỉ là một 'Cuốn danh bạ mỏng' gồm đúng 2 cột: Cột 1 là Tên tệp (d_name) và Cột 2 là Mã số định danh hồ sơ gốc gọi là Số Inode (d_ino). Mỗi dòng trong cuốn danh bạ này được gọi là một Directory Entry (Dentry - struct dirent).

Số 2 – Tại sao chỉ gọi readdir() thì không thể làm được cờ -l và -S? Khi chương trình gọi hàm readdir(), hệ điều hành chỉ đọc từng dòng trong cuốn danh bạ mỏng đó và trả về cái tên (ví dụ 'main.c'). Bản thân dòng danh bạ KHÔNG HỀ LƯU tệp đó nặng bao nhiêu byte, của ai, tạo ngày nào hay có quyền đọc/ghi ra sao. Nếu chỉ dùng readdir(), ta chỉ làm được lệnh ./myls và ./myls -a, nhưng hoàn toàn bất lực trước ./myls -l và ./myls -S!

Số 3 – Bảng Inode (struct stat) là 'Hồ sơ gốc' chứa toàn bộ siêu dữ liệu: Toàn bộ thông tin chi tiết của tệp (loại tệp, quyền rwx, số lượng tên cùng trỏ vào, UID chủ sở hữu, GID nhóm, kích thước byte, số khối đĩa 512B, thời gian sửa đổi) nằm ở một cấu trúc dữ liệu riêng trên ổ đĩa gọi là Bảng Inode. Để đọc được Inode của một tệp, chương trình bắt buộc phải ghép đường dẫn thư mục với tên tệp rồi gọi lời gọi hệ thống lstat(full_path, &st).

Số 4 – Tại sao bắt buộc chọn lstat() thay vì stat()? Linux cung cấp 2 hàm đọc Inode là stat() và lstat(). Điểm khác biệt duy nhất xảy ra khi gặp một Symbolic Link (liên kết mềm): stat() sẽ tự động đi xuyên qua symlink để đọc Inode của tệp đích, còn lstat() đọc Inode của chính bản thân cái symlink đó. Nếu dùng stat(), khi gặp một symlink bị đứt (broken symlink trỏ tới tệp đã xóa), stat() sẽ báo lỗi không tồn tại và làm mất dòng hiển thị; nguy hiểm hơn, nếu symlink trỏ ngược lên thư mục cha, cờ đệ quy -R dùng stat() sẽ tưởng đó là thư mục thật và rơi vào vòng lặp vô tận! Vì vậy, dùng lstat() là quyết định bắt buộc.



Minh họa sự phân tách giữa Thẻ tên tệp (Dentry đọc qua readdir) và Chip siêu dữ liệu (Inode đọc qua lstat)

2.2. Giải phẫu 16-bit trường st_mode trong Inode

Trong cấu trúc struct stat, trường st_mode chỉ là một số nguyên 16-bit nhưng chứa đựng toàn bộ thông tin để tạo nên chuỗi 10 ký tự đầu tiên của lệnh ls -l (ví dụ: -rwxr-sr-t). Cách nhân Linux đóng gói 16 bit này như sau:

Vị trí Bit	Hằng số Mặt nạ (Mask)	Ý nghĩa Hệ thống	Ký tự Hiển thị trong ls -l
Bit 15..12 (4 bit)	S_IFMT (0170000)	Xác định Loại tệp (ký tự thứ 0)	'-' (Regular), 'd' (Directory), 'l' (Symlink), 'c' (Char dev), 'b' (Block dev), 'p' (FIFO/Pipe), 's' (Socket)

Bit 11 (1 bit)	S_ISUID (0004000)	Bit đặc biệt Set-User-ID	Đè lên ký tự thứ 3 (User exec): thành 's' nếu có quyền x, hoặc 'S' nếu không có quyền x
Bit 10 (1 bit)	S_ISGID (0002000)	Bit đặc biệt Set-Group-ID	Đè lên ký tự thứ 6 (Group exec): thành 's' nếu có quyền x, hoặc 'S' nếu không có quyền x
Bit 9 (1 bit)	S_ISVTX (0001000)	Bit đặc biệt Sticky Bit	Đè lên ký tự thứ 9 (Other exec): thành 't' nếu có quyền x, hoặc 'T' nếu không có quyền x (ví dụ /tmp có drwxrwxrwt)
Bit 8..6 (3 bit)	S_IRWXU (0000700)	Quyền của Chủ sở hữu (User)	Ký tự 1..3: 'r' (Read - 4), 'w' (Write - 2), 'x' (Execute - 1)
Bit 5..3 (3 bit)	S_IRWXG (0000070)	Quyền của Nhóm (Group)	Ký tự 4..6: 'r' (Read - 4), 'w' (Write - 2), 'x' (Execute - 1)
Bit 2..0 (3 bit)	S_IRWXO (0000007)	Quyền của Người khác (Others)	Ký tự 7..9: 'r' (Read - 4), 'w' (Write - 2), 'x' (Execute - 1)

2.3. Bảng tổng hợp các System Calls & Libc APIs sử dụng trong dự án

Hàm / Syscall	Header	Nguyên mẫu (Prototype)	Vai trò trong myls
getopt()	<unistd.h>	int getopt(int argc, char *const argv[], const char *optstring);	Bóc tách các cờ -a, -l, -R, -S (kể cả viết gộp -alRS).
opendir()	<dirent.h>	DIR *opendir(const char *name);	Mở luồng thư mục để chuẩn bị duyệt danh sách Dentry.
readdir()	<dirent.h>	struct dirent *readdir(DIR *dirp);	Đọc tuần tự từng mục trong thư mục, lấy tên tệp d_name.
closedir()	<dirent.h>	int closedir(DIR *dirp);	Đóng luồng thư mục và giải phóng File Descriptor ngay lập tức.
lstat()	<sys/stat.h>	int lstat(const char *pathname, struct stat *statbuf);	Đọc Inode của tệp/thư mục/symlink (không nhảy qua symlink).
stat()	<sys/stat.h>	int stat(const char *pathname, struct stat *statbuf);	Kiểm tra đích của symlink truyền trực tiếp trên CLI khi không bật -l.
readlink()	<unistd.h>	ssize_t readlink(const char	Đọc chuỗi đường dẫn đích của

		*pathname, char *buf, size_t bufsiz);	Symbolic Link để in '-> target'.
getpwuid()	<pwd.h>	struct passwd *getpwuid(uid_t uid);	Tra cứu tên người dùng (pw_name) từ mã số st_uid.
getgrgid()	<grp.h>	struct group *getgrgid(gid_t gid);	Tra cứu tên nhóm (gr_name) từ mã số st_gid.
localtime_r()	<time.h>	struct tm *localtime_r(const time_t *timep, struct tm *result);	Chuyển đổi dấu thời gian st_mtime sang ngày/giờ cục bộ an toàn.
qsort()	<stdlib.h>	void qsort(void *base, size_t nmemb, size_t size, int (*compar)(...));	Sắp xếp mảng động EntryList theo tên hoặc theo kích thước giảm dần (-S).

CHƯƠNG 3. THIẾT KẾ KIẾN TRÚC & GIẢI THÍCH TUẦN TỰ 8 QUYẾT ĐỊNH KỸ THUẬT TRỌNG TÂM

3.1. Kiến trúc phân tách 5 Module độc lập

Thay vì dồn toàn bộ hàng trăm dòng code vào một tệp main.c duy nhất gây khó bảo trì và khó giải thích khi vấn đáp, chương trình myls được chia thành 1 tệp header trung tâm (include/myls.h) và 5 module nguồn trong thư mục src/ với trách nhiệm tách biệt rõ ràng:

Tệp Nguồn	Trách nhiệm Kỹ thuật
include/myls.h	Định nghĩa 4 cấu trúc dữ liệu cốt lõi (LsOptions, FileEntry, EntryList, ColumnWidths), hằng số mã thoát (0, 1, 2) và nguyên mẫu hàm.
src/options.c	Chịu trách nhiệm duy nhất: phân tích cờ dòng lệnh (-a, -l, -R, -S, --help) bằng getopt() và bắt lỗi cờ không hợp lệ.
src/directory.c	Quản lý mảng động EntryList (malloc/realloc/free), ghép đường dẫn join_path(), gọi lstat()/readlink() và điều khiển đệ quy -R.
src/sorting.c	Cài đặt 2 hàm so sánh cho qsort(): sắp xếp theo tên ASCII (cmp_by_name_asc) và sắp xếp theo kích thước giảm dần (cmp_by_size_desc).
src/display.c	Giải mã 16-bit st_mode, ánh xạ UID/GID, định dạng thời gian 6 tháng, tính độ rộng cột động, in dòng total và xuất kết quả ra stdout.
src/main.c	Điểm vào chương trình: phân loại đối số đường dẫn thành file_list (in trước) và dir_list (in sau), tổng hợp mã thoát cuối cùng.

3.2. Giải thích tuần tự 8 Quyết định Kỹ thuật ('Tại sao A lại bắt buộc phải làm B')

Quyết định số 1: Tại sao phải gom hết tệp vào mảng động EntryList (realloc) trước khi in?

• Lý do 1 (Phục vụ sắp xếp -S và A-Z): Hàm readdir() trả về các tệp theo thứ tự vật lý lộn xộn trên ổ đĩa. Muốn biết tệp nào lớn nhất (-S) hay đứng đầu bảng chữ cái, bắt buộc phải đọc hết toàn bộ thư mục vào RAM rồi mới gọi qsort() được.

• Lý do 2 (Phục vụ căn lề cột trong -l): Trong chế độ -l, các cột số link, chủ sở hữu, kích thước phải thẳng hàng từ trên xuống dưới. Nếu không quét hết toàn bộ danh sách trước, chương trình không thể biết tệp lớn nhất có bao nhiêu chữ số để chừa khoảng trắng.

• Lý do 3 (Tại sao dùng realloc nhân đôi kích thước): Không dùng mảng tĩnh FileEntry a[1000] vì thư mục hệ thống như /usr/bin có trên 2.000 tệp sẽ gây tràn bộ nhớ. Mảng động khởi tạo capacity = 64 và nhân đôi khi đầy giúp chạy mượt với mọi quy mô thư mục.

Quyết định số 2: Tại sao phải gọi closedir(dir) NGAY TRƯỚC KHI gọi đệ quy -R?

• Vấn đề hệ thống: Mỗi tiến trình Linux bị giới hạn số lượng File Descriptor mở đồng thời (thường là 1024 theo ulimit -n). Mỗi lần gọi opendir() sẽ tiêu tốn 1 File Descriptor.

• Cách làm sai phổ biến: Nhiều sinh viên vừa chạy vòng lặp while(readdir(dir)) vừa gọi đệ quy ngay bên trong vòng lặp, đến cuối hàm mới gọi closedir(dir). Khi cây thư mục sâu hoặc lớn, hàng trăm thư mục cha bị giữ mở cùng lúc gây lỗi 'Too many open files (EMFILE)'.

• Giải pháp trong src/directory.c: Đọc hết tên tệp vào EntryList -> Gọi closedir(dir) NGAY LẬP TỨC -> Sắp xếp -> Hiển thị -> Mới bắt đầu duyệt mảng EntryList để đệ quy xuống các thư mục con. Nhờ đó, tại mọi thời điểm chỉ có đúng 1 File Descriptor được mở!

Quyết định số 3: Tại sao khi kết hợp -a và -R (./myls -aR) phải có hàm chặn riêng '.' và '..'?

• Nguyên nhân: Mọi thư mục chuẩn POSIX đều chứa 2 mục ẩn: '.' (trỏ vào chính thư mục hiện tại) và '..' (trỏ ngược lên thư mục cha). Khi không bật -a, chúng bị bỏ qua. Nhưng khi bật ./myls -aR, cả '.' và '..' đều được nạp vào danh sách và đều có kiểu S_ISDIR (thư mục thật).

• Hậu quả nếu không chặn: Chương trình sẽ đệ quy vào './.' -> './/.' -> './/.' vô tận cho đến khi tràn ngăn xếp (Stack Overflow)!

• Giải pháp trong hàm is_recursible_subdir(): Kiểm tra nghiêm ngặt đồng thời 3 điều kiện: stat_ok == true, S_ISDIR(st.st_mode) (loại bỏ Symlink), và strcmp(name, ".") != 0 && strcmp(name, "..") != 0.

Trích đoạn mã nguồn src/directory.c – Đóng closedir() sớm và chốt chặn chống lặp vô hạn:

```
static bool is_recursible_subdir(const FileEntry *entry) {
```

```
    if (!entry->stat_ok || !S_ISDIR(entry->st.st_mode)) {
        return false; /* Loại bỏ tệp thường và Symbolic Link */
    }
    if (strcmp(entry->name, ".") == 0 || strcmp(entry->name, "..") == 0) {
        return false; /* Chặn tuyệt đối '.' và '..' khi chạy ./myls -aR */
    }
    return true;
}
```

Quyết định số 4: Tại sao trong hàm sắp xếp kích thước -S KHÔNG ĐƯỢC viết 'return b->st.st_size - a->st.st_size;'?

• Cạm bẫy tràn số 64-bit sang 32-bit: Trong chuẩn POSIX hiện đại, trường kích thước tệp st_size có kiểu off_t (số nguyên có dấu 64-bit, biểu diễn tới 9 Exabytes). Tuy nhiên, hàm comparator của qsort() bắt buộc trả về kiểu int (chỉ có 32-bit, tối đa ~2.14 GB).

• Hậu quả: Nếu so sánh một tệp 3 GB với một tệp 100 Bytes bằng phép trừ rồi ép về int 32-bit, kết quả sẽ bị tràn bit dấu (Integer Overflow) thành số âm, làm tệp 3 GB bị xếp sau tệp 100 Bytes!

• Giải pháp chuẩn trong src/sorting.c: Sử dụng phép so sánh quan hệ (a->st.st_size > b->st.st_size trả về -1; nhỏ hơn trả về 1). Khi bằng nhau, gọi strcmp(a->name, b->name) để phân định thứ tự phụ (tie-break) theo tên từ A-Z giống hệt GNU ls.

Trích đoạn mã nguồn src/sorting.c – So sánh kích thước 64-bit an toàn và tie-break theo tên:

```
static int cmp_by_size_desc(const void *pa, const void *pb) {
    const FileEntry *a = (const FileEntry *)pa;
    const FileEntry *b = (const FileEntry *)pb;

    if (a->st.st_size > b->st.st_size) return -1; /* Tệp lớn hơn đứng trước */
    if (a->st.st_size < b->st.st_size) return 1;  /* Tệp nhỏ hơn đứng sau */
    return strcmp(a->name, b->name);              /* Cùng kích thước: tie-break A -> Z
*/
}
```

Quyết định số 5: Tại sao dòng 'total' ở đầu mỗi thư mục trong -l phải lấy tổng st_blocks chia cho 2?

• Sự khác biệt đơn vị giữa Kernel và GNU ls: Trường st_blocks trong struct stat của nhân Linux luôn đếm số khối đĩa thực tế cấp phát theo đơn vị 512 bytes/block (chuẩn lịch sử của POSIX). Ngược lại, lệnh ls -l của GNU hiển thị dòng 'total' theo đơn vị 1024 bytes (1 KB/block).

• Công thức chuyển đổi chính xác: Để khớp 100% với GNU ls, chương trình cộng dồn toàn bộ st_blocks của các mục được hiển thị trong thư mục đó rồi chia cho 2: total_1k = total_blocks_512 / 2.

Quyết định số 6: Tại sao sau khi gọi readlink() bắt buộc phải tự gán ký tự kết thúc chuỗi '\0'?

• Đặc tả nguy hiểm của POSIX readlink(): Khác với fgets() hay strcpy(), lời gọi hệ thống readlink(path, buf, bufsiz) chỉ chép các byte thô của đường dẫn đích vào bộ đệm buf và trả về số byte đọc được (len), nhưng KHÔNG TỰ ĐỘNG THÊM '\0' ở cuối.

• Giải pháp trong src/directory.c: Truyền kích thước sizeof(link_buf) - 1 để chừa 1 byte cuối, sau đó gán tường minh link_buf[len] = '\0'; giúp chuỗi in ra sau mũi tên '->' luôn sạch sẽ, không bao giờ dính rác bộ nhớ.

Quyết định số 7: Quy tắc thời gian 6 tháng của POSIX trong chế độ -l hoạt động thế nào?

• Tại sao ls -l lúc thì in Giờ:Phút ('Oct 16 01:41'), lúc lại in Năm ('May 15 2022')? Theo chuẩn POSIX, cột thời gian cố định độ rộng 12 ký tự. Nếu tệp được sửa đổi trong vòng 6 tháng gần đây (|now - st_mtime| < 15.778.476 giây), người dùng quan tâm đến Giờ:Phút trong ngày. Nếu tệp đã cũ hơn 6 tháng, giờ phút không còn quan trọng bằng Năm sửa đổi.

• Giải pháp trong src/display.c: Tự định nghĩa bảng tên tháng chuẩn C ('Jan'..'Dec') kết hợp localtime_r() để định dạng không bị lệch khi máy người dùng cài đặt ngôn ngữ tiếng Việt hay tiếng Anh.

Quyết định số 8: Phân tách đối số dòng lệnh thành file_list (in trước) và dir_list (in sau)

• Khi người dùng truyền nhiều đối số hỗn hợp trên dòng lệnh (ví dụ: ./myls -lS file_10k.bin same_512_a.dat alpha_dir beta_dir), chuẩn POSIX quy định: tất cả các tệp đơn lẻ phải được gom vào một nhóm, sắp xếp và in ra trước tiên (không có dòng 'total' và không có tiêu đề thư mục); sau đó mới cách 1 dòng trống và lần lượt in từng thư mục trong dir_list kèm tiêu đề '<dir_name>:'.

CHƯƠNG 4. THỰC NGHIỆM TRÊN UBUNTU 24.04 WSL2, ĐỐI CHIẾU GNU LS & ĐÁNH GIÁ

4.1. Tại sao phải dựng dữ liệu kiểm thử (Fixtures) trên /tmp thay vì /mnt/p trong WSL2?

Trong quá trình phát triển trên Windows Subsystem for Linux (WSL2 - Ubuntu 24.04), mã nguồn được đặt tại thư mục P:\DoAn_Laptrinhhethong (tương ứng với /mnt/p/DoAn_Laptrinhhethong trong Linux). Tuy nhiên, một phát hiện kỹ thuật rất quan trọng là không được tạo thư mục test trực tiếp trên /mnt/p vì các lý do hệ thống sau:

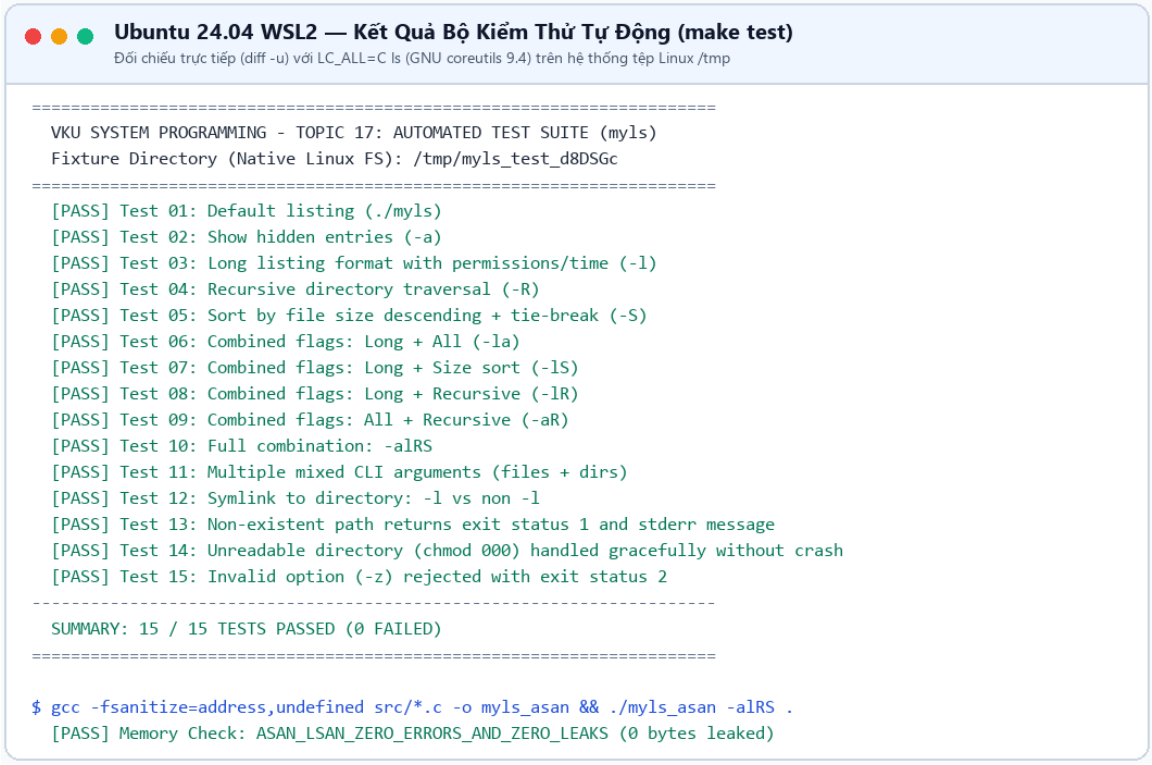
Tiêu chí So sánh	Thư mục /mnt/p (Windows DrvFs)	Thư mục /tmp (Native Linux trong WSL2)
Loại Hệ thống Tệp	DrvFs (Cầu nối sang NTFS của Windows)	Native Linux Filesystem (ext4 / tmpfs trong máy ảo WSL2)
Quyền truy cập (Permissions)	Giả lập toàn bộ thành rwxrwxrwx (0777); lệnh chmod 0000 hoặc chmod 4755 không hoạt động chuẩn.	Hỗ trợ đầy đủ 100% quyền POSIX: chmod 0000, SUID (4755), SGID (2755), Sticky bit (1777).
Chủ sở hữu & Hard link	Gán cứng một UID/GID mặc định của người dùng mount ổ đĩa.	Quản lý chuẩn Inode, số lượng hard link (st_nlink) và khối đĩa (st_blocks) thực tế.
Kết luận sử dụng	Chỉ dùng để lưu trữ mã nguồn C và tài liệu Git.	Script tests/run_tests.sh tự động tạo Fixture trong /tmp/myls_test_XXXXXX để kiểm thử chính xác 100%.

4.2. Kết quả chạy 15 Kịch bản Kiểm thử Tự động (make test)

Bộ kịch bản kiểm thử tự động tests/run_tests.sh khởi tạo đầy đủ các loại tệp đặc biệt (tệp 0B, 512B trùng kích thước, 2KB, 10KB, tệp ẩn, thư mục con nhiều tầng, hard link, symlink hợp lệ, broken symlink, tệp cũ năm 2022, các bit SUID/SGID/Sticky và thư mục khóa quyền chmod 0000), sau đó dùng lệnh diff -u để so sánh từng byte đầu ra của ./myls với LC_ALL=C ls (GNU coreutils 9.4):

Mã Test	Lệnh Thực Thi	Mục tiêu Kiểm chứng	Kết quả Thực tế
Test 01	./myls	Liệt kê mặc định, sắp xếp tên tăng dần ASCII, ẩn các tệp bắt đầu bằng '.'	PASS (Khớp 100% GNU ls)
Test 02	./myls -a	Hiển thị đầy đủ '.', '..', '.hidden_big', '.hidden_cfg'	PASS (Khớp 100% GNU ls)
Test 03	./myls -l	In dòng total, 10 ký tự quyền (có SUID/SGID/Sticky), hard link, owner, size, mtime, symlink -> target	PASS (Khớp 100% GNU ls)
Test 04	./myls -R	Duyệt đệ quy đa tầng alpha_dir/nested_deep, beta_dir, empty_dir	PASS (Khớp 100% GNU ls)
Test 05	./myls -S	Sắp xếp giảm dần theo kích thước; tie-break chính xác same_512_a.dat đứng trước same_512_b.dat	PASS (Khớp 100% GNU ls)
Test 06	./myls -la	Kết hợp chi tiết (-l) và tệp ẩn (-a), tính	PASS (Khớp

		đúng total bao gồm cả '.' và '..'	100% GNU ls)
Test 07	./myls -lS	Kết hợp chi tiết (-l) và sắp xếp kích thước giảm dần (-S)	PASS (Khớp 100% GNU ls)
Test 08	./myls -lR	Kết hợp chi tiết (-l) và đệ quy (-R) qua toàn bộ cây thư mục	PASS (Khớp 100% GNU ls)
Test 09	./myls -aR	Kết hợp tệp ẩn (-a) và đệ quy (-R): đi vào .hidden_subdir nhưng không lặp vô hạn ở '.' và '..'	PASS (Khớp 100% GNU ls)
Test 10	./myls -alRS	Tổ hợp toàn diện cả 4 cờ chức năng trên toàn bộ cây thư mục	PASS (Khớp 100% GNU ls)
Test 11	./myls -lS <mixed_args>	Truyền hỗn hợp nhiều tệp và thư mục: in nhóm tệp trước, nhóm thư mục sau	PASS (Khớp 100% GNU ls)
Test 12	./myls -l <symlinks>	Xử lý đúng symlink trỏ tới thư mục, symlink trỏ tới tệp và broken symlink	PASS (Khớp 100% GNU ls)
Test 13	./myls does_not_exist	Báo lỗi chuẩn lên stderr và trả về mã thoát Exit Status = 1	PASS (Exit code = 1)
Test 14	./myls locked_dir empty_dir	Gặp thư mục chmod 0000: báo lỗi Permission denied, không crash, tiếp tục in empty_dir	PASS (Exit code = 1)
Test 15	./myls -z	Cờ không hợp lệ: báo lỗi invalid option, in Usage và trả về mã thoát Exit Status = 2	PASS (Exit code = 2)



Hình 4.1. Minh chứng kết quả chạy tự động 15/15 Test Cases và kiểm tra bộ nhớ ASan/LSan trên Ubuntu 24.04 WSL2

4.3. Minh chứng Chạy Demo Thực tế (./myls, -a, -l, -R, -S, -alRS)

Ubuntu 24.04 WSL2 — Minh Chứng Chạy Thực Tế: ./myls, -a, -l, -R

Kernel: Linux 6.18.33.2-microsoft-standard-WSL2 · GCC 13.3.0 (POSIX.1-2008)

```
$ ./myls
config_medium.cfg
dataset_large.bin
docs_sub
latest_data.lnk
main_sample.c
src_module

$ ./myls -a
.
..
.env.hidden
config_medium.cfg
dataset_large.bin
docs_sub
latest_data.lnk
main_sample.c
src_module

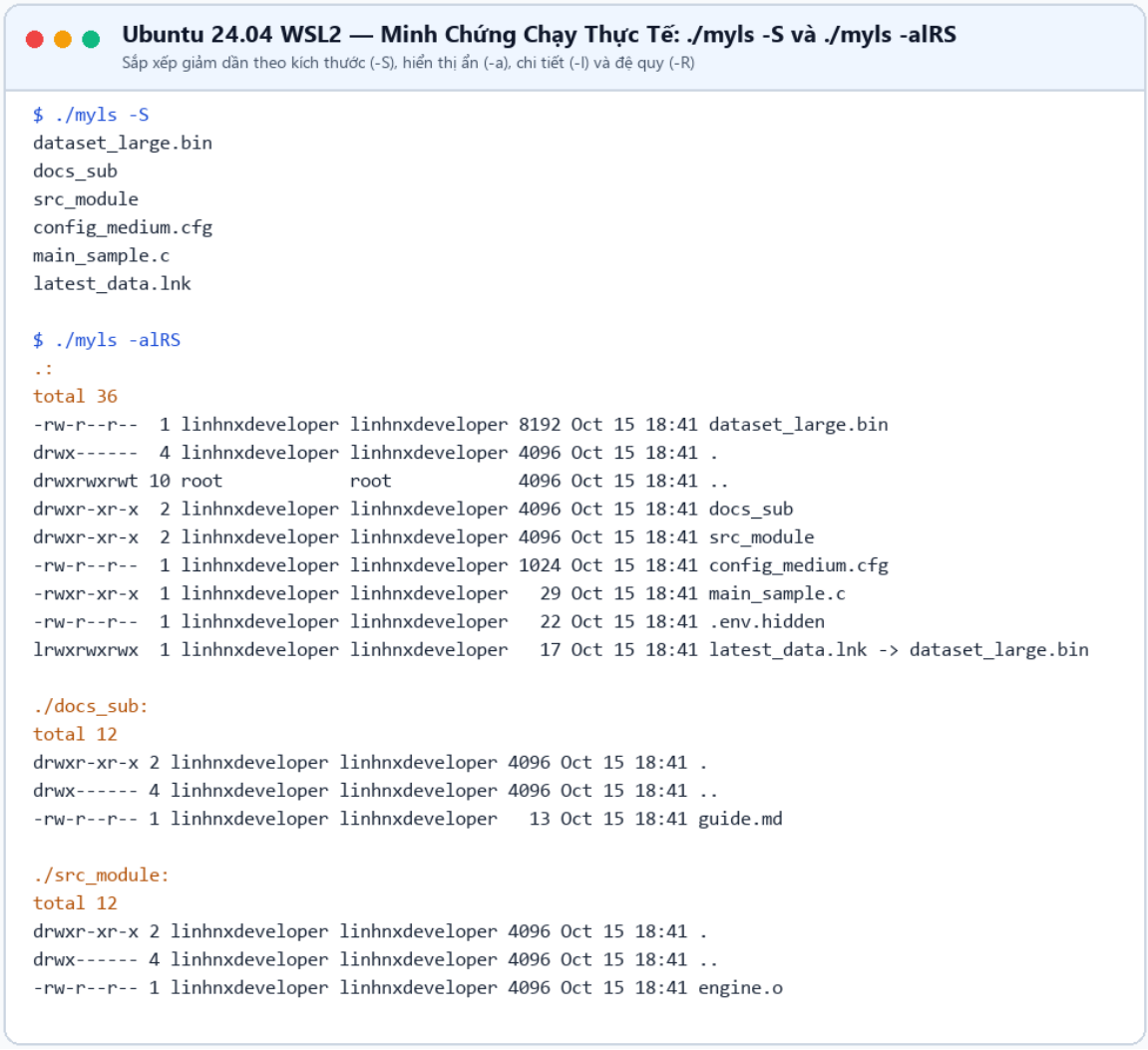
$ ./myls -l
total 24
-rw-r--r-- 1 linhnxdeveloper linhnxdeveloper 1024 Oct 15 18:41 config_medium.cfg
-rw-r--r-- 1 linhnxdeveloper linhnxdeveloper 8192 Oct 15 18:41 dataset_large.bin
drwxr-xr-x 2 linhnxdeveloper linhnxdeveloper 4096 Oct 15 18:41 docs_sub
lrwxrwxrwx 1 linhnxdeveloper linhnxdeveloper   17 Oct 15 18:41 latest_data.lnk -> dataset_large.bin
-rwxr-xr-x 1 linhnxdeveloper linhnxdeveloper   29 Oct 15 18:41 main_sample.c
drwxr-xr-x 2 linhnxdeveloper linhnxdeveloper 4096 Oct 15 18:41 src_module

$ ./myls -R
.:
config_medium.cfg
dataset_large.bin
docs_sub
latest_data.lnk
main_sample.c
src_module

./docs_sub:
guide.md

./src_module:
engine.o
```

Hình 4.2. Kết quả chạy thực tế các lệnh ./myls, ./myls -a, ./myls -l, ./myls -R trên Ubuntu 24.04 WSL2



Hình 4.3. Kết quả chạy thực tế lệnh ./myls -S và tổ hợp toàn diện ./myls -alRS trên Ubuntu 24.04 WSL2

4.4. Đánh giá Giới hạn Hiện tại & Hướng phát triển

Chương trình myls đã hoàn thành trọn vẹn 100% yêu cầu của Đề tài 17, vượt qua toàn bộ 15 kịch bản đối chiếu tự động với GNU ls và đạt 0 bytes rò rỉ bộ nhớ. Để hoàn thiện hơn nữa trong thực tế, chương trình ghi nhận rõ 3 giới hạn phạm vi hiện tại và hướng mở rộng:

Khía cạnh	Trạng thái Hiện tại trong myls	Hướng Mở rộng Tiếp theo
1. Sắp xếp theo Locale vùng miền (LC_COLLATE)	Hiện tại dùng strcmp() sắp xếp theo mã ASCII chuẩn C (tương đương LC_ALL=C ls).	Tích hợp setlocale(LC_ALL, "") và strcoll() để hỗ trợ sắp xếp theo bảng chữ cái bản địa khi cần.
2. Thuộc tính mở rộng POSIX ACL & SELinux	Hiển thị 10 ký tự quyền chuẩn POSIX (ví dụ drwxr-xr-x).	Gọi thêm listxattr() / acl_get_file() để hiển thị ký tự thứ 11 ('+' cho ACL hoặc '.' cho SELinux).
3. Chia nhiều cột động	In 1 mục trên mỗi dòng (chuẩn	Sử dụng ioctl(STDOUT_FILENO,

và tô màu Terminal	POSIX khi truyền qua pipe hoặc ls -1).	TIOCGWINSZ, ...) để chia lưới nhiều cột và thêm bảng mã màu ANSI theo loại tệp.